



ساختمان داده‌ها و الگوریتم‌ها

تمرین اول - پیچیدگی و الگوریتم‌های بازگشتی

آریا عازم، مهدی یاری

تاریخ تحویل: ۱۴۰۴/۰۷/۲۹

۲۵ نمره

۱. پیچیدگی زمانی

پیچیدگی زمانی کدهای زیر را بررسی کنید. (در صورت تفاوت بین بهترین و بدترین حالت، هر دو را بیان کنید.)

- a)

```
int func1 (int n) {  
    int cnt = 0;  
    for (int i = 1; i < n; i *= 2) {  
        for (int j = 0; j < n - i; j++) {  
            cnt++;  
        }  
    }  
    return cnt;  
}
```
- b)

```
void func2 (int n) {  
    int i = n;  
    while (i > 0) {  
        int j = 1;  
        while (j < i) {  
            j *= 2;  
        }  
        i--;  
    }  
}
```
- c)

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j <= i; j++)  
        for (int k = 0; k < n - j; k++){  
            sum += k;  
        }
```
- d)

```
mergeCheck(A, B, m, n) {  
    int i = 0, j = 0;  
    while (i < m && j < n) {  
        if (A[i] == B[j])  
            i++, j++;  
        else if (A[i] < B[j])  
            i++;  
        else  
            j++;  
    }
```

```

    }
    return (i + j);
}

e) int func5(int n) {
    int x = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j += 2) {
            x += (i + j);
        }
    }
    return x;
}

```

۲. روابط بازگشتی

۲۰ نمره

پیچیدگی روابط بازگشتی زیر را با استفاده از روش‌های ممکن (قضیه اصلی، درخت بازگشتی، یا تبدیل متغیر) به دست آورید و ایده حل را به صورت واضح توضیح دهید.

- $T(n) = 3T(n-1) + O(1)$
- $T(n) = 16T\left(\frac{n}{4}\right) + n^2$
- $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n$
- $T(n) = T(\sqrt{n}) + T(n^{1/3}) + O(\log n)$

۳. مرتب‌سازی توابع

۲۰ نمره

با ذکر راه حل، توابع را بر حسب درجه رشد مرتب کنید.

$\sqrt{n}^2$, $\log_2(n)$, $50n$, $n^{3/4}$, $n^{1/100}$, $\log(n!)$, n^n , 5^n , 5^{n+1} , 10^n , 3^n , $n \log n$, $256n^{2/3}$, $\log((\log n)^5)$, 777^{777} , $n!$, $(n-2)!$

۴. تابع بازگشتی عجیب

۱۵ نمره

برای تابع زیر:

```

void weirdo(int n)
{
    if (n <= 1) return;
    int L = floor(log2n)

    for (int k = 1; k <= L; k++) {
        weirdo(n >> k);
    }

    for (int i = 0; i < n; i++)
        do_constant_work();
}

```

}

الف) رابطه بازگشتی زمان اجرا را بنویسید.

 ب) این رابطه را حل کنید و مرتبه زمانی $T(n)$ را به دست آورید.

 ج) اگر به جای حلقه دوم، یک $\Theta(1)$ کار انجام می‌شد، چه تغییری در پاسخ ایجاد می‌شد؟

۵. اثبات یا رد روابط

۱۰ نمره

روابط زیر را با استفاده از تعریف (اثبات مستقیم) یا با مثال نقض رد کنید. در صورت صحیح بودن اثبات ریاضی داده و در غیر این صورت مثال نقض بزنید.

- a) $O(f \cdot g) = O(f) \cdot O(g)$
- b) *if* $g = O(f)$ *then* $g \cdot n = O(f)$
- c) *if* $g = O(f)$ *then* $g = O(f + n)$
- d) *if* $f = \Theta(g)$ *and* $g = \Theta(h)$ *then* $f = \Theta(h)$
- e) *if* $f = o(g)$ *then* $g = O(f)$

۶. عملکرد تابع بازگشتی

۱۰ نمره

توضیح دهید تابع زیر چه عملیاتی انجام می‌دهد، سپس رابطه بازگشتی زمان اجرای آن را بنویسید، مرتبه زمانی را محاسبه کنید و تعداد مقایسه‌های عناصر آرایه را (دقیق یا کران بالا) محاسبه و اثبات کنید.

```
typedef struct { long value; int freq; } Res;

Res function(long arr[], int size)
{
    if(size == 1) {
        Res r; r.value = arr[0]; r.freq = 1;
        return r;
    }

    int mid = size / 2;

    Res v1 = function(arr + mid, size - mid);
    Res v2 = function(arr, mid);

    if (v1.value > v2.value)
        return v1;
    else if (v1.value < v2.value)
        return v2;
    else {
        Res r; r.value = v1.value; r.freq = v1.freq + v2.freq;
        return r;
    }
}
```